

UNIVERSITY OF DENVER
Department of Electrical and Computer Engineering
LABORATORY REPORT

COURSE NO: ENCE 4110

COURSE NAME: Modern Digital Systems Design

Instructor: Dr. Goncalo Martins

Submitted by:
Monowar Mahmud
DU ID: 873827421

1. Introduction

This laboratory exercise centers on the implementation of the basic combinational logic circuits using Verilog HDL programming language on the DE10-Lite FPGA development board. The task starts from switch-to-LED interfacing followed by the implementation of 2-to-1 and multiple input multiplexers (MUXs). These components are then combined with the seven-segment decoder to form characters such as H, E, L, and O. Next, hierarchical multiplexer and decoder are implemented to show and circularly rotate the HELLO word using the available seven-segment displays. The experiment also gives an opportunity for working with the RTL and synthesized circuit diagrams in the Intel Quartus Prime environment.

2. Hardware and Software

The laboratory exercises were performed on the DE10-Lite FPGA development board and Intel Quartus Prime. In Verilog HDL, combinational logic circuit was described, synthesized and implemented. For control input, DE10-Lite on-board slide switches were used and for output, red LEDs and seven-segment displays were used. The hardware and software summary is presented in Table 1.

Table 1: Table 1. Hardware and Software Used in the Laboratory

Component	Description
FPGA Board	DE10-Lite
FPGA Inputs	10 slide switches, SW[9:0]
LED Outputs	10 red LEDs, LEDR[9:0]
Seven-Segment Displays	Six displays, HEX5–HEX0
Development Software	Intel Quartus Prime
Hardware Description Language	Verilog HDL

3. Experimental Design and Results

3.1 Part I — Switch-to-LED Interface

The first test of this experiment was performed to check the fundamental I/O functionality of the DE10-Lite board. There was a direct connection between the ten switch controls SW[9:0] and red LEDs, LEDR[9:0] through the following continuous assignment in Verilog:

```
assign LEDR = SW;
```

In this arrangement, each LED reflects the logic value of the respective switch. If there was a high logic level at the input of the switch, then the LED was turned on; otherwise, if the logic value of the input was low, the LED was turned off.

Verilog Code:

```
module main(  
    input [9:0] SW,  
    output [9:0] LEDR,  
);
```

```
// PART I
```

```
assign LEDR = SW;
```

```
endmodule
```

3.2 Part II — 1-Bit 2-to-1 Multiplexer

In Part II, a **1-bit 2-to-1 multiplexer** was implemented using Verilog HDL. The multiplexer has two data inputs, x and y ; and a select input, s . The output m is determined by:

$$m = \bar{s}x + sy$$

Therefore, x is selected when $s=0$, while y is selected when $s=1$. For the DE10-Lite implementation, SW[0] was used as the select input, SW[1] and SW[2] were the two data inputs, and LEDR[0] was used as the output.

As shown in Fig. 1, the RTL schematic diagram shows the realization of the MUX using a combinational logic that involves an inverter, two AND, and one OR gates. Testing was done for various switch configurations, where LEDR[0] followed the chosen input.

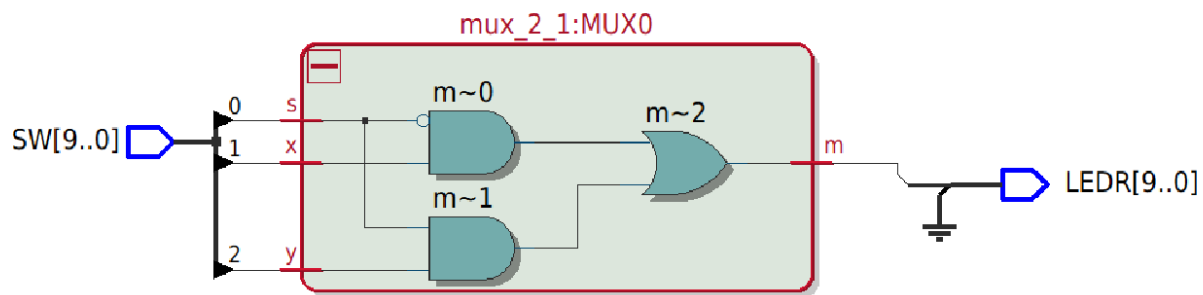


Fig. 1. RTL schematic of the 1-bit 2-to-1 multiplexer

Verilog Code:

```
module mux_2_1 (  
    input s,  
    input x,  
    input y,  
    output m  
);  
    assign m = (~s & x) | (s & y);  
endmodule
```

```
module part2 (  
    input [9:0] SW,
```

```

    output [9:0] LEDR
);

```

```

mux_2_1 MUX0 (
    .s(SW[0]),
    .x(SW[1]),
    .y(SW[2]),
    .m(LED0[0])
);

```

```
endmodule
```

3.3 Part III — Multiplexer Expansion

3.3.1 8-Bit 2-to-1 Multiplexer

The 1-bit multiplexer from Part II was used as a basis for the construction of the 8-bit 2-to-1 multiplexer. There is a total of eight 1-bit multiplexers working in parallel with a shared select signal, and only one out of two 8-bit inputs can be passed through to the output. Hence, the hierarchical RTL design consists of eight 1-bit multiplexers.

In the implementation of the design using the DE10-Lite board, the select signal SW[0], the first 8-bit input SW[8:1], and the second 8-bit input decimal 170 (8'b10101010) were used. The output 8-bit value was shown on LEDR[7:0].

The RTL diagram depicted in Fig. 2 demonstrates that the eight 1-bit multiplexers are indeed implemented in parallel. This design was verified using the FPGA, and the output LEDs responded to the 8-bit input correctly.

Verilog Code:

```

module mux_2_1_8_bit (
    input    s,
    input [7:0] x,
    input [7:0] y,
    output [7:0] m
);

```

```

module mux_2_1_8_bit (
    input s,
    input [7:0] x,
    input [7:0] y,

```

```
output [7:0] m  
);
```

```
mux_2_1 MUX0 (  
    .s(s),  
    .x(x[0]),  
    .y(y[0]),  
    .m(m[0])  
);
```

```
mux_2_1 MUX1 (  
    .s(s),  
    .x(x[1]),  
    .y(y[1]),  
    .m(m[1])  
);
```

```
mux_2_1 MUX2 (  
    .s(s),  
    .x(x[2]),  
    .y(y[2]),  
    .m(m[2])  
);
```

```
mux_2_1 MUX3 (  
    .s(s),  
    .x(x[3]),  
    .y(y[3]),  
    .m(m[3])  
);
```

```
mux_2_1 MUX4 (  
    .s(s),
```

```
.x(x[4]),  
.y(y[4]),  
.m(m[4])  
);
```

```
mux_2_1 MUX5 (  
    .s(s),  
    .x(x[5]),  
    .y(y[5]),  
    .m(m[5])  
);
```

```
mux_2_1 MUX6 (  
    .s(s),  
    .x(x[6]),  
    .y(y[6]),  
    .m(m[6])  
);
```

```
mux_2_1 MUX7 (  
    .s(s),  
    .x(x[7]),  
    .y(y[7]),  
    .m(m[7])  
);
```

```
endmodule
```

```
module part3_8bit (  
    input [9:0] SW,  
    output [9:0] LEDR  
);  
    mux_2_1_8_bit MUX0 (  
        .s(SW[0]),
```

```
.x(SW[8:1]),  
.y(8'd170),  
.m(LED[7:0])  
);
```

```
endmodule
```

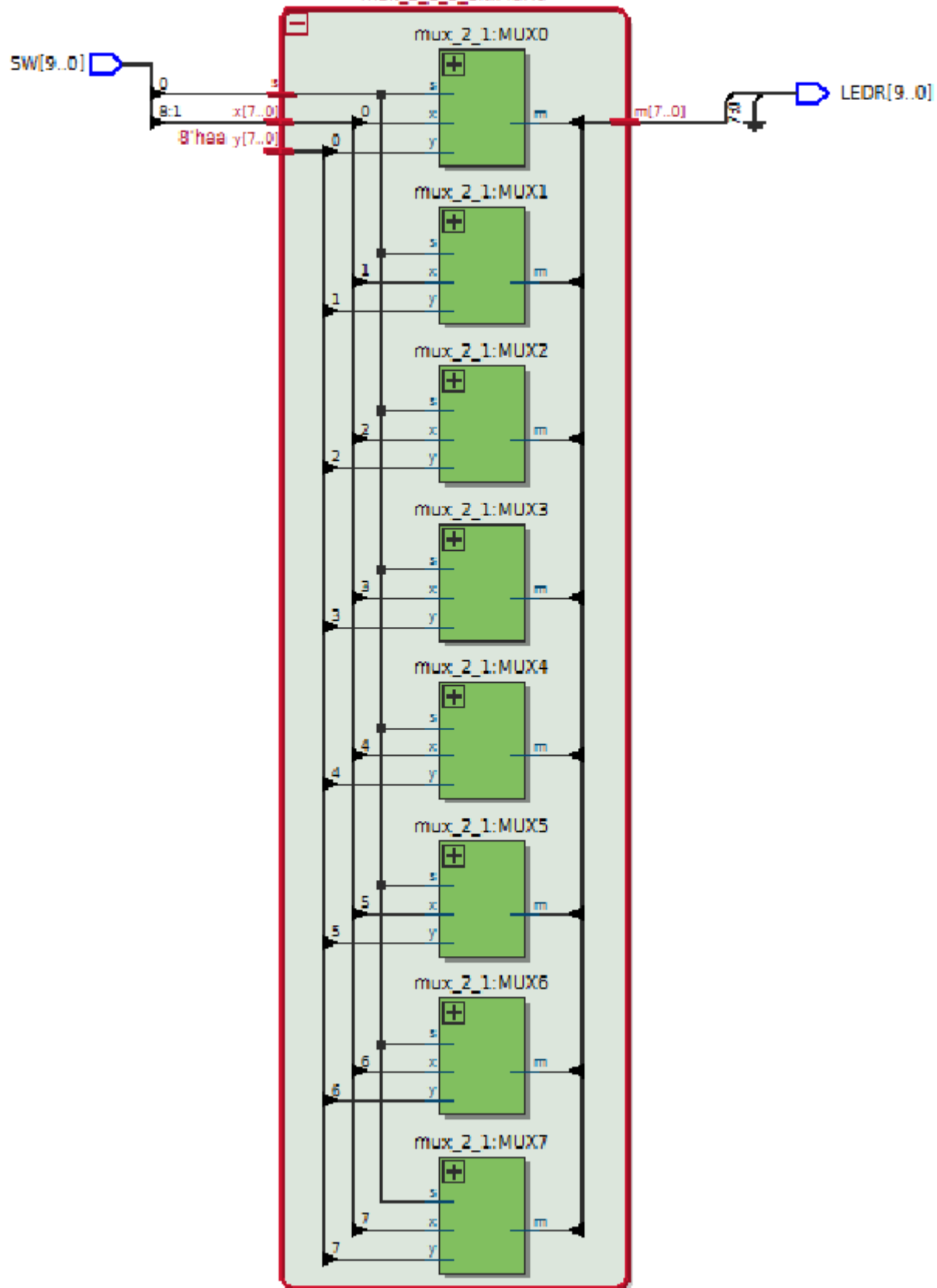


Fig. 2. RTL schematic of the 8-bit 2-to-1 multiplexer

3.3.2 3-Bit 5-to-1 Multiplexer

A **3-bit-wide 5-to-1 multiplexer** was implemented using the hierarchical multiplexer structure. Five 3-bit inputs, U, V, W, X, and Y, are selected using the 3-bit control input S. The input selection is summarized in Table 2.

Table 2: Input selection of the 3-bit 5-to-1 multiplexer

S	Selected Input
000	U
001	V
010	W
011	X
100–111	Y

In DE10-Lite implementation, SW[2:0] was used as select. The two inputs were received using SW[5:3] and SW[8:6], whereas the other three were assigned constant value. Hierarchical Implementation by the RTL diagram of Fig. 3 uses different levels of 2-to-1 Multiplexers. Testing the FPGA implementation with different values of selector produced a display of the selected 3-bit inputs on LEDR[2:0].

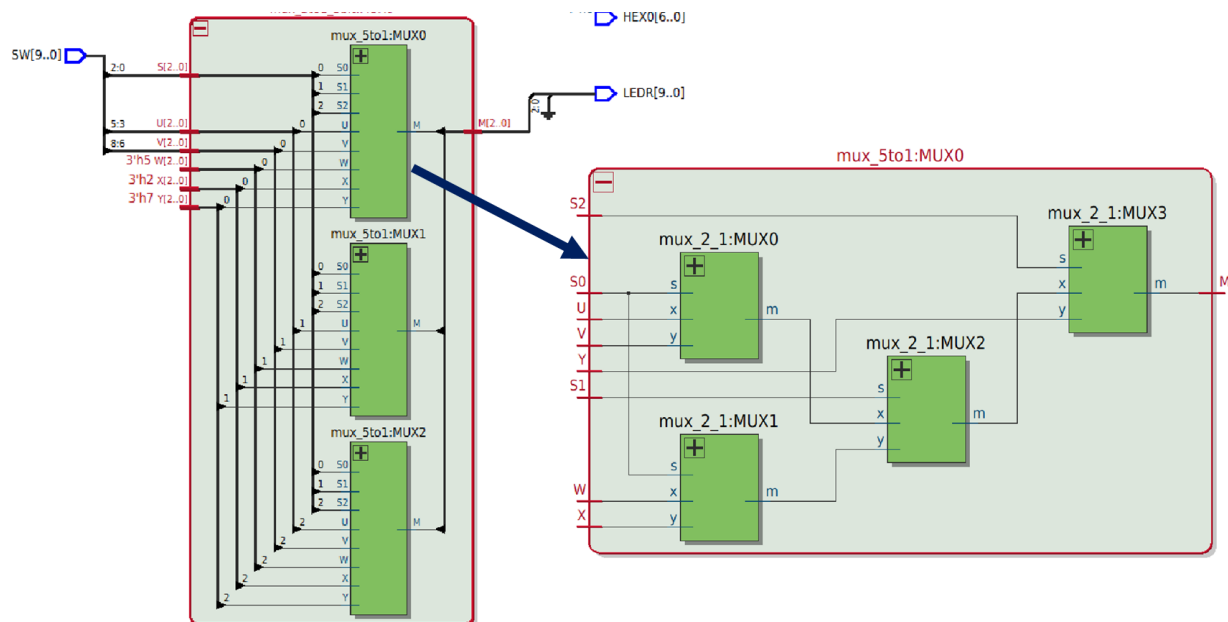


Fig. 3. RTL schematic of the 3-bit 5-to-1 multiplexer

Verilog Code:

```
module mux_5to1 (
    input [2:0] S,
    input U,
    input V,
    input W,
    input X,
```

```
input Y,  
output M  
);
```

```
wire A, B, C;
```

```
// First level
```

```
mux_2_1 MUX0 (  
    .s(S[0]),  
    .x(U),  
    .y(V),  
    .m(A)  
);
```

```
mux_2_1 MUX1 (  
    .s(S[0]),  
    .x(W),  
    .y(X),  
    .m(B)  
);
```

```
// Second level
```

```
mux_2_1 MUX2 (  
    .s(S[1]),  
    .x(A),  
    .y(B),  
    .m(C)  
);
```

```
// Final level
```

```
mux_2_1 MUX3 (  
    .s(S[2]),  
    .x(C),
```

```
.y(Y),  
.m(M)  
);
```

```
endmodule
```

```
module mux_5to1_3bit (  
    input [2:0] S,  
    input [2:0] U,  
    input [2:0] V,  
    input [2:0] W,  
    input [2:0] X,  
    input [2:0] Y,  
    output [2:0] M  
);
```

```
    mux_5to1 MUX0 (  
        .S(S),  
        .U(U[0]),  
        .V(V[0]),  
        .W(W[0]),  
        .X(X[0]),  
        .Y(Y[0]),  
        .M(M[0])  
    );
```

```
    mux_5to1 MUX1 (  
        .S(S),  
        .U(U[1]),  
        .V(V[1]),  
        .W(W[1]),  
        .X(X[1]),  
        .Y(Y[1]),
```

```

        .M(M[1])
    );

    mux_5to1 MUX2 (
        .S(S),
        .U(U[2]),
        .V(V[2]),
        .W(W[2]),
        .X(X[2]),
        .Y(Y[2]),
        .M(M[2])
    );

endmodule

module part3 (
    input  [9:0] SW,
    output [9:0] LEDR
);

    mux_5to1_3bit MUX0 (
        .S(SW[2:0]),
        .U(SW[5:3]),
        .V(SW[8:6]),
        .W(3'b101),
        .X(3'b010),
        .Y(3'b111),
        .M(LEDR[2:0])
    );

endmodule

```

3.4 Part IV — Seven-Segment Character Decoder

In Part IV, a **3-bit to seven-segment decoder** was implemented to display the characters **H, E, L, and O**. The 3-bit input codes represent each letter, while the rest of the codes have no output, as seen in Table 3.

Table 3. Character encoding for the seven-segment decoder

Input	Displayed Character
000	H
001	E
010	L
011	O
100–111	Blank

Since the seven-segment displays are low-active devices in the DE10-Lite, a logic zero activates a segment while a logic one deactivates the segment. The Boolean equations were applied to control each of the seven segments based on the chosen character. Figure 4 illustrates the RTL schematic that was obtained from the combinational logic of the seven-segment decoder design. This design was implemented on the FPGA, and the letters H, E, L, O, and blank were displayed for the given codes.

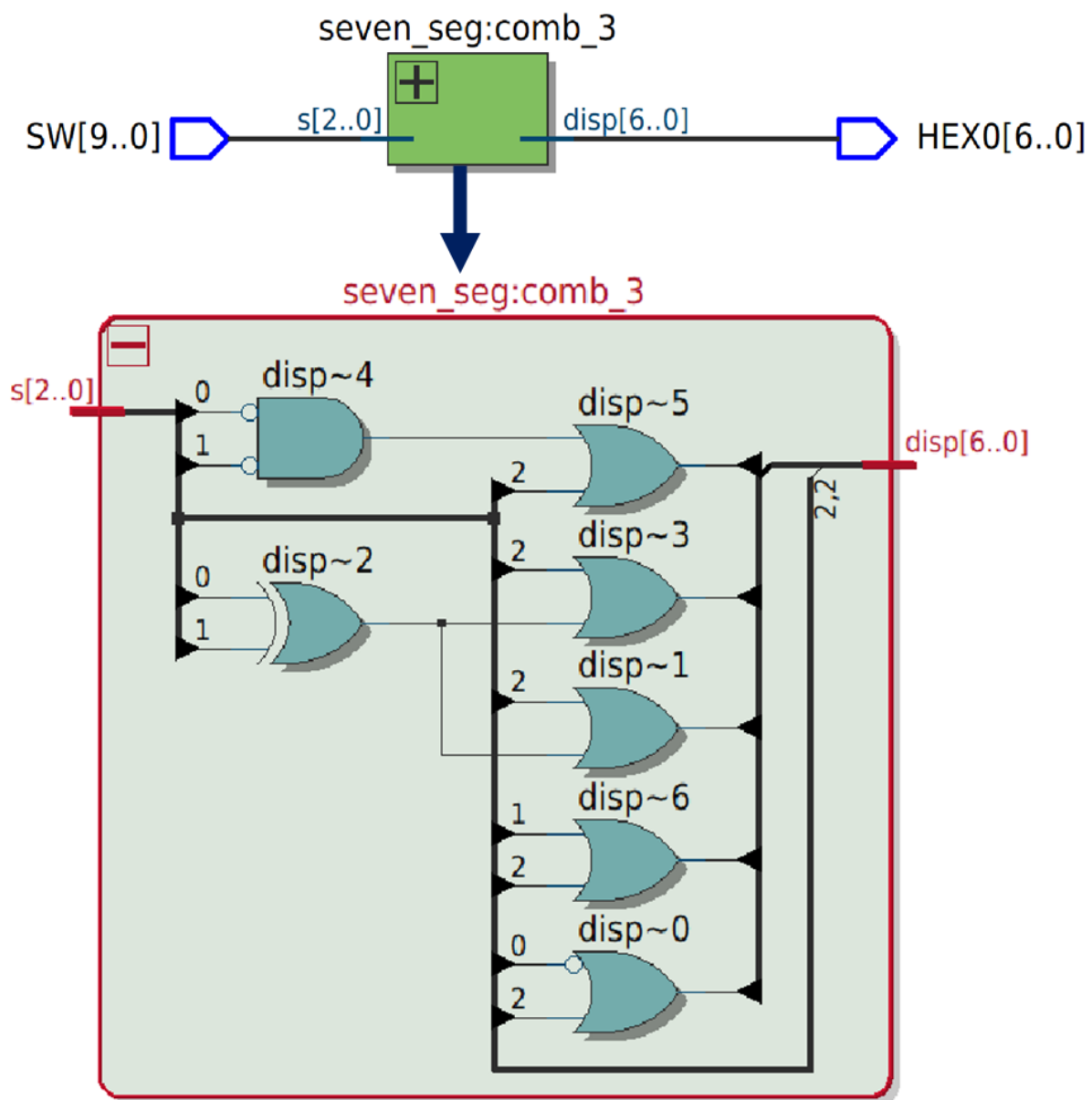


Fig. 4. RTL schematic of the seven-segment character decoder.

Verilog Code:

```
module seven_seg (
    input  [2:0] s,
    output [6:0] disp
);
```

```

    assign disp[0] = s[2] | ~s[0];
    assign disp[1] = s[2] | (s[1] ^ s[0]);
    assign disp[2] = s[2] | (s[1] ^ s[0]);
    assign disp[3] = s[2] | (~s[1] & ~s[0]);
    assign disp[4] = s[2];
    assign disp[5] = s[2];
    assign disp[6] = s[2] | s[1];

endmodule

module part4 (
    input [9:0] SW,
    output [6:0] HEX0
);

    seven_seg Display1 (
        .s(SW[2:0]),
        .disp(HEX0[6:0])
    );

endmodule

```

endmodule

3.5 Part V — Five-Display Character Rotation

Part V involves combining the 3-bit 5-to-1 multiplexer created in Part III with the seven-segment decoder created in Part IV to display the word HELLO on five 7-segment displays and then rotate it. The letters were encoded using 3-bits as follows: U=H, V=E, W=L, X=L, and Y=O. A total of five 3-bit 5-to-1 multiplexers were employed and each of them was connected to the seven-segment decoder. Each of the multiplexers had its input shifted by one digit as shown in Table 4.

Table 4. Character rotation for the five-display implementation

Selector	HEX4 → HEX0
000	HELLO
001	ELLOH
010	LLOHE
011	LOHEL
100	OHELL

The five multiplexer-decoder channels in parallel were employed to provide an output to HEX4 to HEX0 as shown in the RTL diagram in Fig. 5. It was observed during FPGA simulation that circular shift occurs when the selectors are changed.

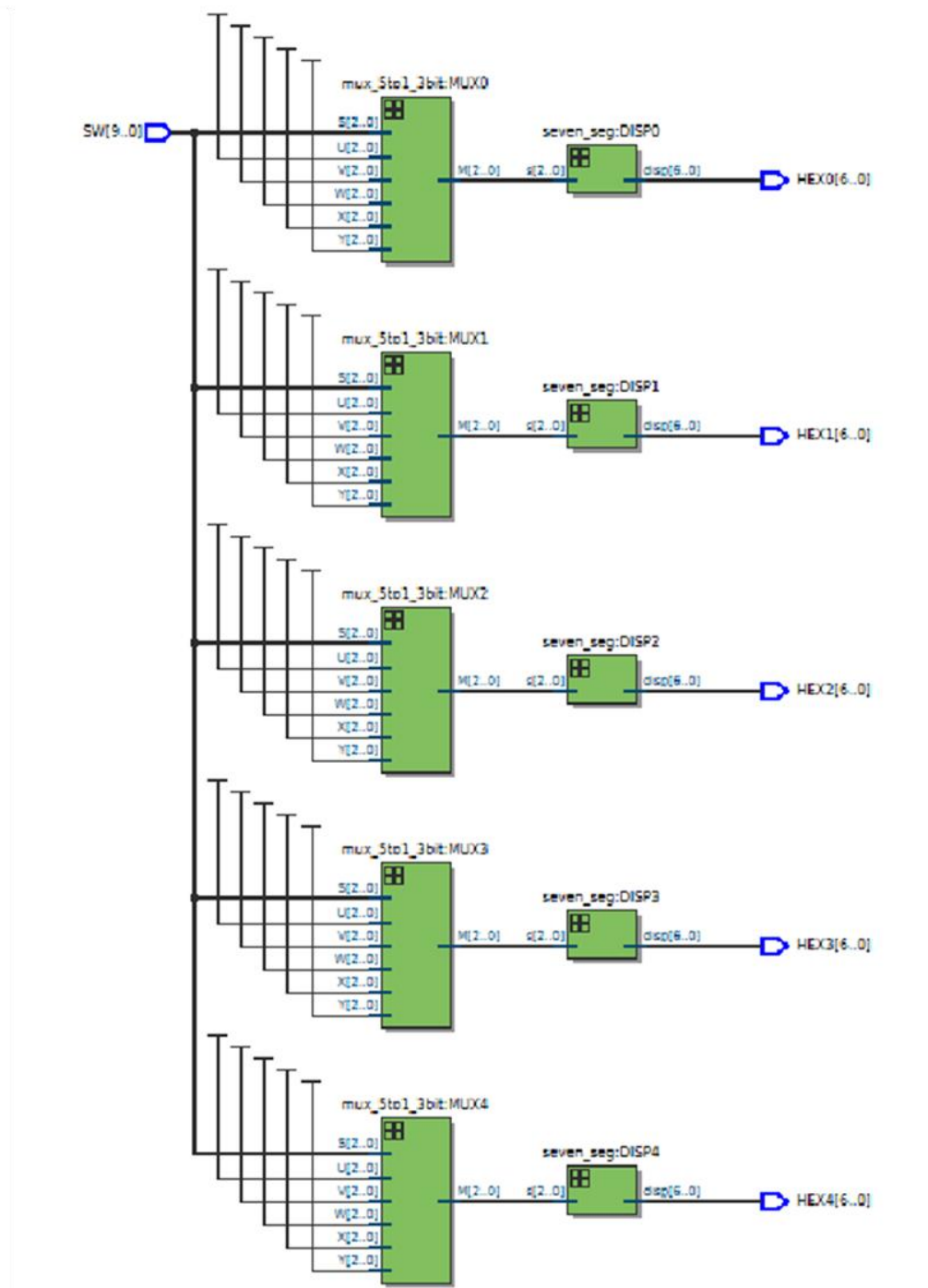


Fig. 5. RTL schematic of the five-display HELLO rotation circuit.

Verilog Code:

module part5 (

```
input [9:0] SW,

output [6:0] HEX0,
output [6:0] HEX1,
output [6:0] HEX2,
output [6:0] HEX3,
output [6:0] HEX4
);
wire [2:0] U;
wire [2:0] V;
wire [2:0] W;
wire [2:0] X;
wire [2:0] Y;

assign U = 3'b000; // H
assign V = 3'b001; // E
assign W = 3'b010; // L
assign X = 3'b010; // L
assign Y = 3'b011; // O

wire [2:0] M0;
wire [2:0] M1;
wire [2:0] M2;
wire [2:0] M3;
wire [2:0] M4;
mux_5to1_3bit MUX4 (
    .S(SW[2:0]),
    .U(U),
    .V(V),
    .W(W),
    .X(X),
    .Y(Y),
    .M(M4)
```

);

```
seven_seg DISP4 (  
    .s(M4),  
    .disp(HEX4[6:0])  
);
```

```
mux_5to1_3bit MUX3 (  
    .S(SW[2:0]),  
    .U(V),  
    .V(W),  
    .W(X),  
    .X(Y),  
    .Y(U),  
    .M(M3)  
);
```

```
seven_seg DISP3 (  
    .s(M3),  
    .disp(HEX3[6:0])  
);
```

```
mux_5to1_3bit MUX2 (  
    .S(SW[2:0]),  
    .U(W),  
    .V(X),  
    .W(Y),  
    .X(U),  
    .Y(V),  
    .M(M2)  
);
```

```
seven_seg DISP2 (  
    .s(M2),  
    .disp(HEX2[6:0])  
);
```

```

        .s(M2),
        .disp(HEX2[6:0])
    );

mux_5to1_3bit MUX1 (
    .S(SW[2:0]),
    .U(X),
    .V(Y),
    .W(U),
    .X(V),
    .Y(W),
    .M(M1)
);

seven_seg DISP1 (
    .s(M1),
    .disp(HEX1[6:0])
);

mux_5to1_3bit MUX0 (
    .S(SW[2:0]),
    .U(Y),
    .V(U),
    .W(V),
    .X(W),
    .Y(X),
    .M(M0)
);

seven_seg DISP0 (
    .s(M0),
    .disp(HEX0[6:0])
);

```

```
endmodule
```

3.6 Part VI — Six-Display Character Rotation

In part VI, the character rotation circuit was further modified to include all six seven-segment displays present in the DE10-Lite board. Apart from the five characters of HELLO, an extra character of a space was also included, making the display sequence HELLO . This circuit was controlled using six 3-bit multiplexers and six seven-segment decoders for controlling HEX5 to HEX0. The order of inputs of each multiplexer was cyclically shifted by one place, thus making the value of SW[2:0] rotate all six characters and one blank. The desired sequence of outputs is presented in Table 5.

Table 5. Character rotation for the six-display implementation

Selector	HEX5 → HEX0
000	HELLO_
001	ELLO_H
010	LLO_HE
011	LO_HEL
100	O_HELL
101	_HELLO

Fig. 6 displays the RTL diagram showing the six parallel mux/decoder lines connected to HEX5 to HEX0. The implementation on FPGA confirmed that the word “HELLO” and the empty location were circularly shifted using the six HEX displays depending on the selector signal.

Verilog Code:

```
module mux_6to1 (  
    input [2:0] S,  
    input A,  
    input B,  
    input C,  
    input D,  
    input E,  
    input F,  
    output M  
);  
  
    assign M = (S == 3'b000) ? A :  
               (S == 3'b001) ? B :  
               (S == 3'b010) ? C :
```

```
(S == 3'b011) ? D :  
(S == 3'b100) ? E :  
(S == 3'b101) ? F :  
    1'b0;
```

```
endmodule
```

```
module mux_3bit_6to1 (  
    input [2:0] S,  
    input [2:0] A,  
    input [2:0] B,  
    input [2:0] C,  
    input [2:0] D,  
    input [2:0] E,  
    input [2:0] F,  
    output [2:0] M  
);
```

```
// Bit 0
```

```
mux_6to1 MUX0 (  
    .S(S),  
    .A(A[0]),  
    .B(B[0]),  
    .C(C[0]),  
    .D(D[0]),  
    .E(E[0]),  
    .F(F[0]),  
    .M(M[0])  
);
```

```
// Bit 1
```

```
mux_6to1 MUX1 (  
    .S(S),
```

```
.A(A[1]),  
.B(B[1]),  
.C(C[1]),  
.D(D[1]),  
.E(E[1]),  
.F(F[1]),  
.M(M[1])  
);
```

```
// Bit 2
```

```
mux_6to1 MUX2 (  
    .S(S),  
    .A(A[2]),  
    .B(B[2]),  
    .C(C[2]),  
    .D(D[2]),  
    .E(E[2]),  
    .F(F[2]),  
    .M(M[2])  
);
```

```
endmodule
```

```
module part6_1 (  
    input [9:0] SW,  
  
    output [6:0] HEX0,  
    output [6:0] HEX1,  
    output [6:0] HEX2,  
    output [6:0] HEX3,  
    output [6:0] HEX4,  
    output [6:0] HEX5  
);
```

```
// Character Codes
```

```
wire [2:0] U;
```

```
wire [2:0] V;
```

```
wire [2:0] W;
```

```
wire [2:0] X;
```

```
wire [2:0] Y;
```

```
wire [2:0] BLANK;
```

```
assign U    = 3'b000; // H
```

```
assign V    = 3'b001; // E
```

```
assign W    = 3'b010; // L
```

```
assign X    = 3'b010; // L
```

```
assign Y    = 3'b011; // O
```

```
assign BLANK = 3'b100; // Blank
```

```
// Multiplexer Outputs
```

```
wire [2:0] M0;
```

```
wire [2:0] M1;
```

```
wire [2:0] M2;
```

```
wire [2:0] M3;
```

```
wire [2:0] M4;
```

```
wire [2:0] M5;
```

```
// HEX5: H E L L O _
```

```
mux_3bit_6to1 MUX5 (
```

```
    .S(SW[2:0]),
```

```
    .A(U),
```

```
    .B(V),
```

```
    .C(W),
```

```
    .D(X),
```

```
    .E(Y),
```

```
    .F(BLANK),
```

```
    .M(M5)
```

```
);
```

```
seven_seg Disp_5 (
```

```
    .s(M5),
```

```
    .disp(HEX5[6:0])
```

```
);
```

```
// HEX4: E L L O _ H
```

```
mux_3bit_6to1 MUX4 (
```

```
    .S(SW[2:0]),
```

```
    .A(V),
```

```
    .B(W),
```

```
    .C(X),
```

```
    .D(Y),
```

```
    .E(BLANK),
```

```
    .F(U),
```

```
    .M(M4)
```

```
);
```

```
seven_seg Disp_4 (
```

```
    .s(M4),
```

```
    .disp(HEX4[6:0])
```

```
);
```

```
// HEX3: L L O _ H E
```

```
mux_3bit_6to1 MUX3 (
```

```
    .S(SW[2:0]),
```

```
    .A(W),
```

```
    .B(X),
```

```
    .C(Y),
```

```
    .D(BLANK),
```

```
    .E(U),
```

```
.F(V),  
.M(M3)  
);
```

```
seven_seg Disp_3 (  
.s(M3),  
.disp(HEX3[6:0])  
);
```

```
// HEX2: L O _ H E L  
mux_3bit_6to1 MUX2 (  
.S(SW[2:0]),  
.A(X),  
.B(Y),  
.C(BLANK),  
.D(U),  
.E(V),  
.F(W),  
.M(M2)  
);
```

```
seven_seg Disp_2 (  
.s(M2),  
.disp(HEX2[6:0])  
);
```

```
// HEX1: O _ H E L L  
mux_3bit_6to1 MUX1 (  
.S(SW[2:0]),  
.A(Y),  
.B(BLANK),  
.C(U),  
.D(V),
```

```
.E(W),  
.F(X),  
.M(M1)  
);
```

```
seven_seg Disp_1 (  
    .s(M1),  
    .disp(HEX1[6:0])  
);
```

```
// HEX0: _ H E L L O  
mux_3bit_6to1 MUX0 (  
    .S(SW[2:0]),  
    .A(BLANK),  
    .B(U),  
    .C(V),  
    .D(W),  
    .E(X),  
    .F(Y),  
    .M(M0)  
);
```

```
seven_seg Disp_0 (  
    .s(M0),  
    .disp(HEX0[6:0])  
);
```

```
endmodule
```

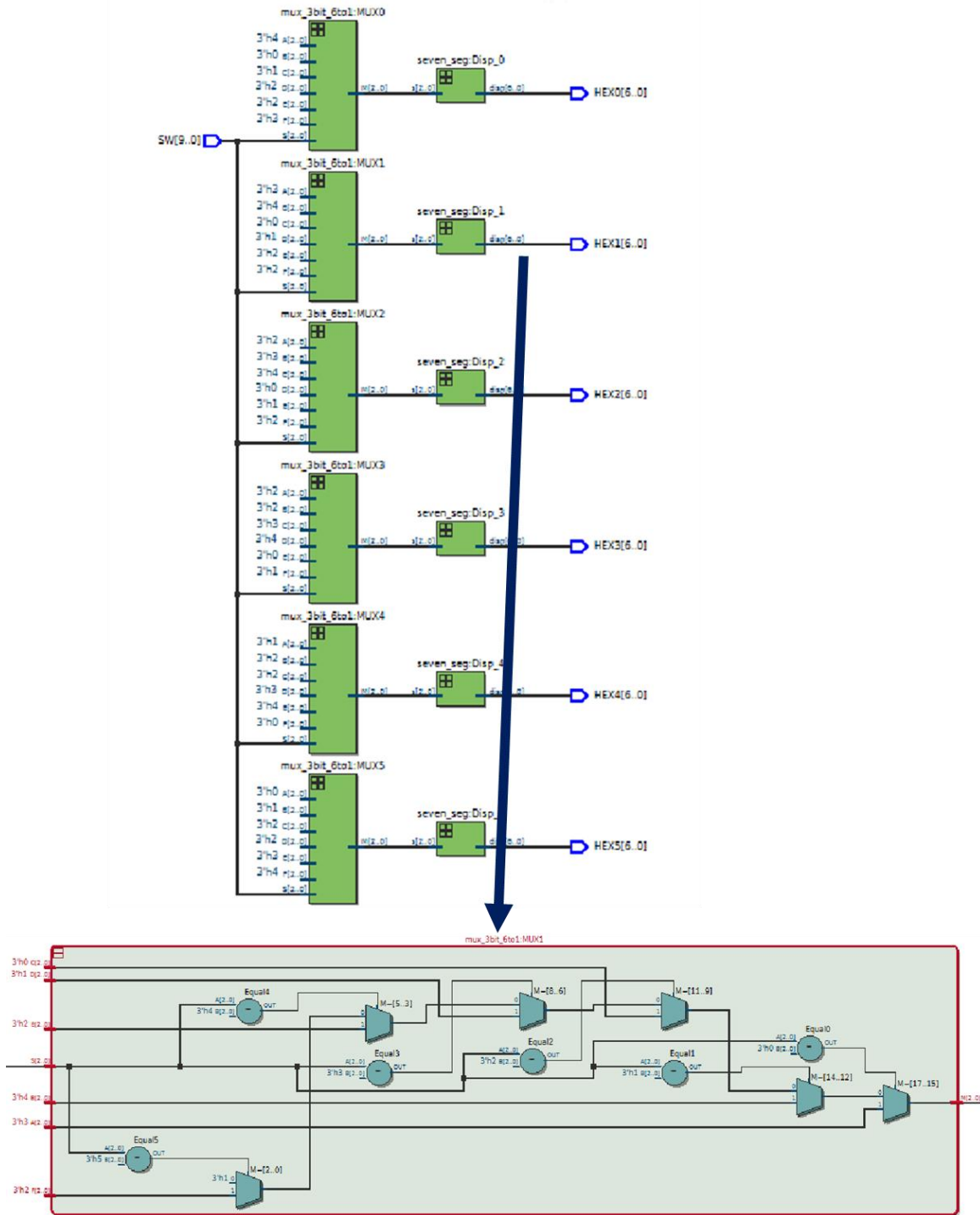


Fig. 6. RTL schematic of the six-display HELLO rotation circuit.

3.7 Automatic Character Rotation Using Clock

The six-display rotation circuit designed was further refined so that character rotations were automated. In place of using SW[2:0] as the multiplexer select input, the 50 MHz internal clock (MAX10_CLK1_50) from the DE10-Lite board was used to drive the circuit for rotation. As the 50 MHz internal clock is too fast for the character rotation observation, a 26-bit clock divider was

used. It divided the number of clock pulses down by 50,000,000 and output one pulse every second to form the slow_tick signal. The slow_tick signal was used to increment the 3-bit rotation counter. The rotation counter counts up from 000 to 101 before going back to 000. The six different states from the counter served as six selections for character rotation. Reset for the two counters was done using SW[9].

The rotation_count signal acted as the replacement for the manually controlled SW[2:0]. Thus, the six multiplexers automatically rotated through the six character positions listed in Table 6.

Table 6. Automatic character rotation sequence

rotation_count	HEX5 → HEX0
000	HELLO_
001	ELLO_H
010	LLO_HE
011	LO_HEL
100	O_HELL
101	_HELLO

The RTL schematic shown in Fig. 7 illustrates the use of additional sequential circuits in order to enable the automatic rotation. A 50 MHz clock pulse drives the clock counter, which was used to create the slow timing pulse. This timing pulse drove the rotation counter, from which 3-bit rotation_count was fed to the select lines of all six multiplexers. The output of each multiplexer was then decoded by the corresponding seven-segment decoder to light up HEX5 through HEX0. In the FPGA design, it was proved that the word HELLO and blank rotate automatically at approximately one-second intervals.

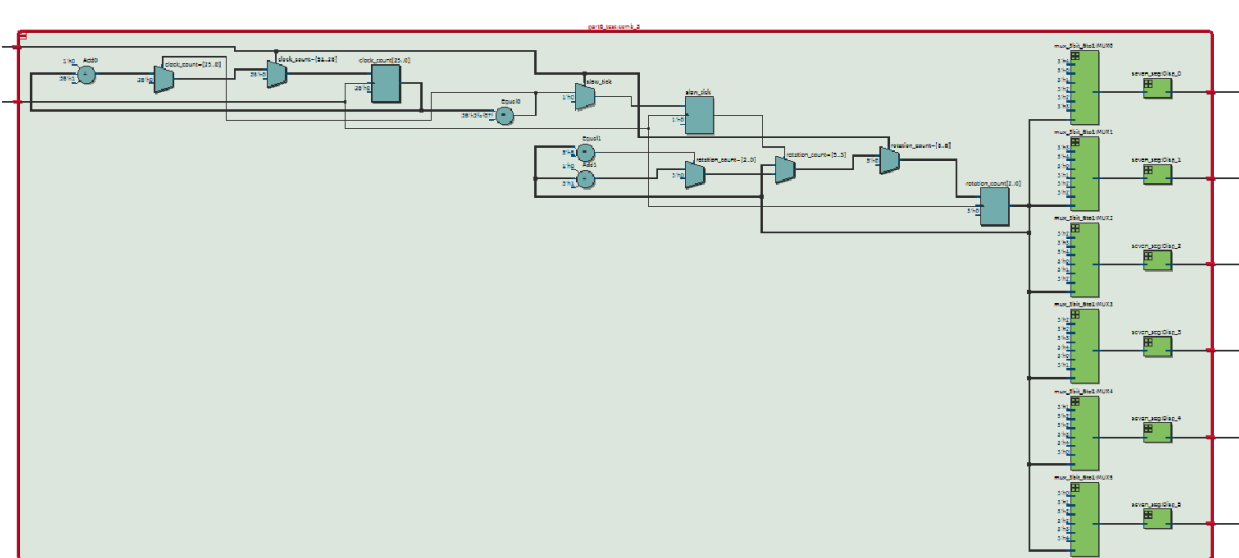


Fig. 7. RTL schematic of the automatic six-display HELLO rotation circuit using the clock divider and rotation counter

Verilog Code

```
module part6_2(
```

```

input MAX10_CLK1_50,
input [9:0] SW,
output [6:0] HEX0,
output [6:0] HEX1,
output [6:0] HEX2,
output [6:0] HEX3,
output [6:0] HEX4,
output [6:0] HEX5
);

// Character codes
wire [2:0] U, V, W, X, Y, BLANK;

assign U    = 3'b000; // H
assign V    = 3'b001; // E
assign W    = 3'b010; // L
assign X    = 3'b010; // L
assign Y    = 3'b011; // O
assign BLANK = 3'b100;

// Clock divider
reg [25:0] clock_count;
reg slow_tick;

always @(posedge MAX10_CLK1_50)
begin
    if (SW[9])
    begin
        clock_count <= 26'd0;
        slow_tick  <= 1'b0;
    end
    else

```

```

begin
    if (clock_count == 26'd49_999_999)
        begin
            clock_count <= 26'd0;
            slow_tick  <= 1'b1;
        end
    else
        begin
            clock_count <= clock_count + 1'b1;
            slow_tick  <= 1'b0;
        end
    end
end
end

```

// Rotation counter

```
reg [2:0] rotation_count;
```

```
always @(posedge MAX10_CLK1_50)
```

```
begin
```

```
    if (SW[9])
```

```
        rotation_count <= 3'b000;
```

```
    else if (slow_tick)
```

```
        begin
```

```
            if (rotation_count == 3'b101)
```

```
                rotation_count <= 3'b000;
```

```
            else
```

```
                rotation_count <= rotation_count + 1'b1;
```

```
        end
```

```
end
```

```
// MUX outputs
wire [2:0] M0, M1, M2, M3, M4, M5;
```

```
// HEX5
mux_3bit_6to1 MUX5 (
    rotation_count,
    U, V, W, X, Y, BLANK,
    M5
);
```

```
seven_seg Disp_5 (
    .s(M5),
    .disp(HEX5)
);
```

```
// HEX4
mux_3bit_6to1 MUX4 (
    rotation_count,
    V, W, X, Y, BLANK, U,
    M4
);
```

```
seven_seg Disp_4 (
    .s(M4),
    .disp(HEX4)
);
```

```
// HEX3
mux_3bit_6to1 MUX3 (
    rotation_count,
```

```
    W, X, Y, BLANK, U, V,  
    M3  
);
```

```
seven_seg Disp_3 (  
    .s(M3),  
    .disp(HEX3)  
);
```

```
// HEX2  
mux_3bit_6to1 MUX2 (  
    rotation_count,  
    X, Y, BLANK, U, V, W,  
    M2  
);
```

```
seven_seg Disp_2 (  
    .s(M2),  
    .disp(HEX2)  
);
```

```
// HEX1  
mux_3bit_6to1 MUX1 (  
    rotation_count,  
    Y, BLANK, U, V, W, X,  
    M1  
);
```

```
seven_seg Disp_1 (  
    .s(M1),  
    .disp(HEX1)
```

```
);
```

```
// HEX0
```

```
mux_3bit_6to1 MUX0 (  
    rotation_count,  
    BLANK, U, V, W, X, Y,  
    M0
```

```
);
```

```
seven_seg Disp_0 (  
    .s(M0),  
    .disp(HEX0)  
);
```

```
endmodule
```